

Generalização e herança em UML e Java



Generalização em UML

A generalização é um conceito importante no Design de Software, especialmente nos diagramas de classes em UML (Unified Modeling Language). Ela estabelece uma relação hierárquica entre classes, permitindo que uma classe mais geral, chamada de superclasse, compartilhe seus atributos e comportamentos com classes mais específicas, chamadas de subclasses.

Conceito de Generalização

Na generalização, a superclasse contém características comuns que podem ser reutilizadas pelas subclasses, que, por sua vez, podem estender ou modificar essas características. Essa estrutura hierárquica é indicada no diagrama de classes por uma seta vazada que aponta da subclasse para a superclasse. O principal objetivo da generalização é reutilizar código e organizar as classes de maneira mais eficiente, representando corretamente o domínio do problema.

Exemplo Prático

Vamos ilustrar com um exemplo:

Imagine uma superclasse chamada *Animal*, que contém atributos comuns como nome e idade, e um método chamado *fazerSom()*. As subclasses *Cachorro* e *Gato* herdam esses atributos e métodos, mas cada uma pode ter suas particularidades. No caso de “*Cachorro*”, o método *fazerSom()* pode ser implementado para latir, enquanto na classe “*Gato*”, ele pode ser implementado para miar.

Diagrama de Classes UML

No diagrama, teríamos a seguinte representação:

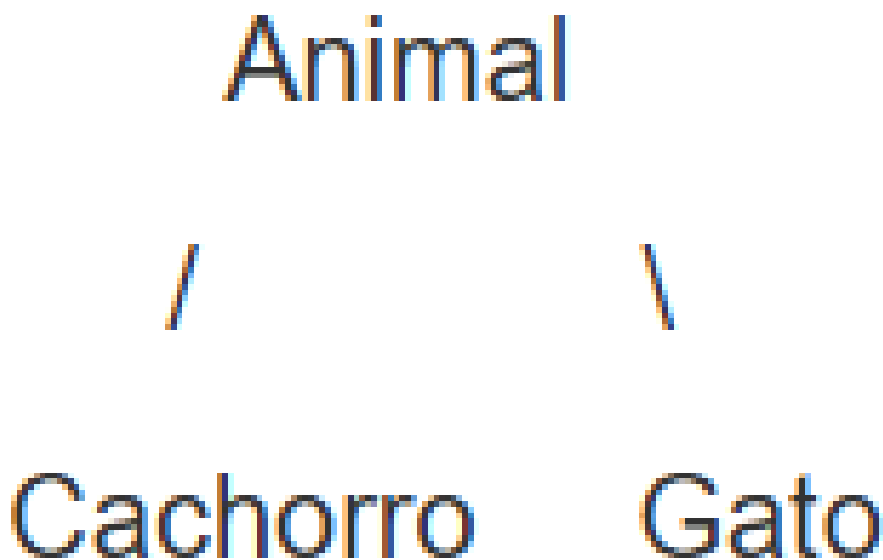
- A superclasse **Animal** no topo.
- Abaixo dela, as subclasses **Cachorro** e **Gato**.
- Uma seta vazada conectando as subclasses à superclasse.

Isso indica que **Cachorro** e **Gato** herdam os atributos e comportamentos da classe **Animal**.

Vantagens da Generalização

A **reutilização de código** é a principal vantagem da generalização. Quando criamos uma superclasse com funcionalidades comuns, evitamos a duplicação de código nas subclasses. Isso facilita a manutenção e a evolução do software, pois qualquer mudança feita na superclasse é automaticamente refletida nas subclasses, reduzindo o esforço e a possibilidade de erros.

Exemplo Visual



Neste diagrama simplificado, vemos que as classes **Cachorro** e **Gato** herdam da classe **Animal**, e a seta vazada indica a relação de herança. A estrutura visual facilita a compreensão da hierarquia entre as classes.

Criação de classes no Astah

Ao abrir o Astah, começamos criando um diagrama de classes. Primeiramente, criamos uma superclasse chamada Animal, que servirá de base para outras classes mais específicas. Isso pode ser feito através do menu de diagrama de classes, e com um duplo clique podemos adicionar um novo elemento de classe.

Em seguida, criamos duas subclasses: Cachorro e Gato, que herdam os atributos e métodos da superclasse Animal. No Astah, é fácil organizar os elementos no diagrama, alinhando-os de forma clara.

Representação da Herança

Para representar a herança, utilizamos a seta de generalização, que é uma linha sólida com a ponta triangular vazada. Conectamos as subclasses Cachorro e Gato à superclasse Animal, indicando que ambas herdam seus atributos e comportamentos. Essa visualização facilita o entendimento da relação entre as classes.

Exemplo de Herança Multinível

Também podemos criar heranças mais complexas, como a herança multinível. Por exemplo, podemos adicionar uma classe intermediária, Mamífero, que herda de Animal, e as classes Cachorro e Gato herdam de Mamífero. Assim, tanto Mamífero quanto suas subclasses herdam os atributos e métodos de Animal, como nome, idade e mover(), mas cada uma pode ter métodos específicos, como latir() para o Cachorro e mear() para o Gato.

Adicionando Interfaces

Além da herança de classes, o Astah permite a criação de interfaces. Podemos criar uma interface chamada ICaça, contendo métodos como perseguir(). As classes Cachorro e Gato podem implementar essa interface, indicando que ambas realizam esse comportamento. No diagrama UML, isso é indicado por uma linha pontilhada conectando a classe à interface.

Generalização em Java

Agora, veremos como a generalização e a herança são implementadas em Java.

Implementando Herança

Em Java, a herança é representada pela palavra-chave `extends`. Ao invés de usar setas, como em UML, o código Java usa essa palavra para indicar que uma classe herda de outra. Por exemplo:

```
class Cachorro extends Animal {  
  
    void latir() {  
  
        System.out.println("O cachorro está latindo");  
  
    }  
  
}
```

Neste exemplo, a classe `Cachorro` herda os atributos e métodos da classe `Animal`.

Criando uma Classe Abstrata

Quando não queremos que uma superclasse seja instanciada, usamos uma **classe abstrata**. Para isso, utilizamos a palavra-chave **abstract**:

```
abstract class Animal {  
  
    String nome;  
  
    int idade;  
  
}
```

Classes como **Cachorro** e **Gato** podem herdar dessa classe, mas a classe **Animal** não pode ser instanciada diretamente.

Sobrescrita de Métodos

A **sobrescrita de métodos** ocorre quando uma subclasse redefine um método da superclasse. Isso permite que o mesmo método tenha comportamentos diferentes em subclasses distintas. Veja o exemplo de sobrescrita do método **mover()**:

```
class Gato extends Animal {  
  
    @Override  
  
    void mover() {  
  
        System.out.println("O gato está caminhando");  
  
    }  
  
}
```

Uso de Interfaces

Outra forma de generalização em Java é o uso de **interfaces**. Interfaces definem métodos que as classes devem implementar. Por exemplo, a interface **IVeiculo**:

```
interface IVeiculo {  
  
    void ligar();  
  
    void desligar();  
  
}
```

A classe **Carro** pode implementar essa interface:

```
class Carro implements IVeiculo {  
  
    @Override  
  
    public void ligar() {  
  
        System.out.println("O carro está ligado");  
  
    }  
  
}
```

```
}
```

```
}
```

Esses são uns dos principais mecanismos de generalização em Java.

Geração de código Java no Astah

Nesta etapa do curso, vamos aprender como gerar o código Java a partir de um diagrama de classes UML no Astah, aplicando o conceito de generalização. A ferramenta Astah nos permite criar diagramas e, automaticamente, gerar o código-fonte Java com base nesse modelo.

Criando o Diagrama de Classes

Vamos começar abrindo o Astah e criando um diagrama de classes. Neste exemplo, teremos uma interface `Animal` que declara os métodos abstratos `falar()` e `mover()`. As classes `Mamífero` e `Ave` vão implementar essa interface. Ambas terão atributos comuns, como `nome` e `idade`, e também métodos próprios: `Mamífero` terá o método `alimentarFilhotes()` e `Ave` o método `botarOvos()`.

Agora, criamos classes especializadas: `Morcego`, que herda de `Mamífero`, e `Pinguim`, que herda de `Ave`. O `Morcego` terá um atributo específico, `envergadura das asas`, e métodos próprios como `usarEcolocalização()`. Já o `Pinguim` terá o método `nadar()`.

Gerando o Código Java

Após montar o diagrama no Astah, utilizamos a funcionalidade de geração de código. No menu `Tools`, selecionamos `Java` para exportar as classes criadas. O Astah gera automaticamente os arquivos `.java` correspondentes ao diagrama.

Exemplo de Código Gerado

Interface `Animal`: contém a declaração dos métodos abstratos `falar()` e `mover()`.

```
public interface Animal {  
  
    void falar();  
  
    void mover();  
  
}
```

Classe Mamífero: implementa a interface Animal e define atributos e métodos herdados pelas subclasses.

```
public class Mamífero implements Animal {  
  
    private String nome;  
  
    private int idade;  
  
    @Override  
  
    public void falar() {  
  
        // Implementação do método falar  
  
    }  
  
    @Override  
  
    public void mover() {  
  
        // Implementação do método mover  
  
    }  
  
    public void alimentarFilhotes() {
```

```
// Método específico da classe Mamífero
```

```
}
```

```
}
```

Classe Morcego: herda de Mamífero e implementa seus próprios métodos.

```
public class Morcego extends Mamífero {
```

```
    private double envergaduraAsas;
```

```
    @Override
```

```
    public void falar() {
```

```
        // Implementação específica para Morcego
```

```
}
```

```
    @Override
```

```
    public void mover() {
```

```
        // Implementação específica para Morcego
```

```
}
```

```
    public void usarEcolocalização() {
```

```
        // Método específico de Morcego
```

```
}
```

```
}
```

Visualizando no Android Studio

Após gerar os arquivos Java, podemos abri-los em um editor como o Android Studio. No ambiente de desenvolvimento, podemos visualizar as interfaces, classes, e a herança implementada, verificando como o código reflete diretamente o modelo criado no Astah.

O uso do Astah agiliza o processo de desenvolvimento, permitindo que o diagrama de classes se converta em código Java de maneira rápida e precisa.

Riscos no uso de herança

A herança e a generalização são ferramentas poderosas para reutilização de código, facilitando a manutenção e legibilidade do software. No entanto, o mau uso desses conceitos pode criar mais problemas do que soluções.

Um dos erros mais comuns ao utilizar herança é criar subclasses que não adicionam nada de novo além de um nome diferente. Se uma subclasse não tem comportamentos ou atributos diferentes da superclasse, a herança não faz sentido. Nesses casos, a subclasse não deveria existir, pois não há especialização real. O objetivo da herança é compartilhar comportamentos comuns, mas também permitir que subclasses adicionem características específicas.

Princípio de Substituição de Liskov (LSP)

Um dos princípios mais importantes para evitar problemas com herança é o Princípio de Substituição de Liskov (LSP). Ele afirma que subclasses devem poder substituir suas superclasses sem alterar o comportamento correto do programa. Isso significa que o código que utiliza uma superclasse deve funcionar da mesma maneira se for utilizada uma instância de uma subclasse.

Se uma subclasse não se comporta de forma consistente com sua superclasse, isso é um indicativo de que a herança foi aplicada de forma inadequada. O Princípio de Substituição de Liskov é um dos pilares do conjunto de práticas de design conhecido como SOLID (S = Single Responsibility, O = Open-Closed, L = Liskov Substitution, I = Interface Segregation, D = Dependency Inversion).

Cuidados ao Utilizar Herança

Para evitar os riscos do uso indevido de herança, é importante se questionar se:

A subclasse realmente especializa a superclasse, ou está apenas replicando seus atributos e métodos?

O código que utiliza a superclasse consegue funcionar corretamente com instâncias da subclasse?

Aplicar herança de forma descuidada pode comprometer a manutenibilidade e a clareza do código, tornando-o mais complexo do que deveria ser.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Como fazer herança em Java e generalização com UML - Parte 01”, que está disponível no canal Kane Chan Dev no YouTube. Nesse vídeo, são trabalhados os conceitos de herança em Java utilizando a programação orientada a objetos. Além de programar, o vídeo também aborda a interpretação de diagramas de classe UML, oferecendo uma abordagem prática e teórica para reforçar o que foi discutido nesta aula.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. Fundamentos da programação de computadores. 3.ed. Pearson: 2012.

BRAGA, P. H. Teste de software. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). Arquitetura de software. Pearson: 2017.

GALLOTTI, G. M. A. Qualidade de software. Pearson: 2015.

MEDEIROS, E. Desenvolvendo software com UML 2.0 definitivo. Pearson: 2004.

MORAIS, I. S. de. (Org.). Engenharia de software. Pearson: 2017.

PFLEEGER, S. L. Engenharia de software: teoria e prática. 2.ed. Pearson: 2003.

SOMMERVILLE, I. Engenharia de software. 10.ed. Pearson: 2019.